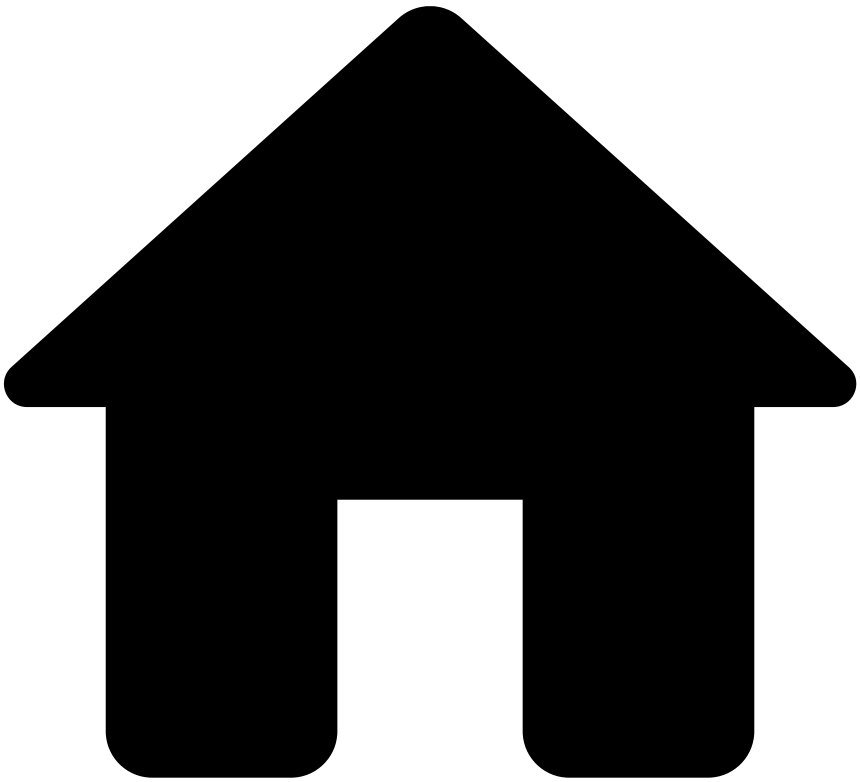


Semantic file

To give meaning to input data fields in an AutoML run, you need to provide a semantic file that tags the data fields to their meaning. The following sections describe how to create the semantic file, and give a list of tags you can use for input fields.

Why tagging?â□□

When defining an AutoML run, you provide a semantic file that maps the input data fields to semantic tags. The semantic tags translate the meaning of the fields, providing the only portion of human knowledge that is required to power AutoML.

If AutoML knows the meaning of the input fields, it can combine them into features using map and metric operators. For example, the *Name* field can be tagged with the *name* tag. This way AutoML knows the meaning of this input field, and it can compute the number of words in the name, which can be useful to fight fraud.

Without tagging the input data fields, AutoML cannot generate features. The quality of the models generated by AutoML relies heavily on the quality of tagging.

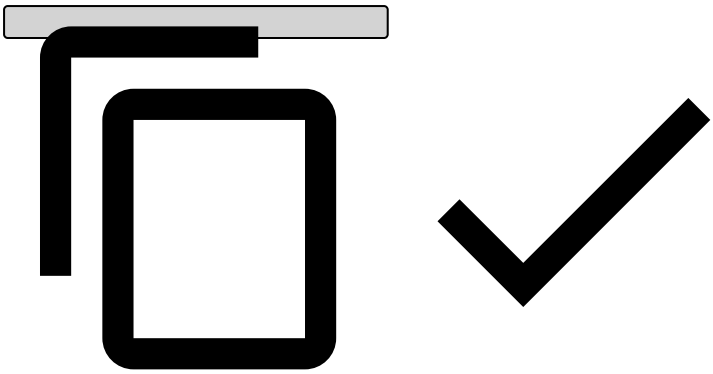
Creating the semantic fileâ□□

The semantic file is a JSON file, in which each data field is an object with the following information:

- **@type**: A string containing the type of the tag, which can be *"simple"* or *"composite"*. Use the **composite tagging** to compose one or more tagged input fields into a single tag. By default, the *address* and *coordinate* tags are composite. See their description in the list of tags.
- **tags**: A vector of strings with the tag(s). See their description in the list of tags.
- **fieldDesc**: A string with the visible name of the field. In *composite* tags it is optional. If you provide a fieldDesc, it will be used as the name of the new field. Otherwise AutoML generates a field name by concatenating the tags you provided.

The following is a simple semantic file, for a data source with four data fields. The last tag creates a composite tag over two fields.

```
[
    {
        "@type": "simple",
        "tags": ["name"],
        "fieldDesc": "Name"
    },
    {
        "@type": "simple",
        "tags": ["amount"],
        "fieldDesc": "TransactionAmount"
    },
    {
        "@type": "simple",
        "tags": ["longitude"],
        "fieldDesc": "TransactionLongitude"
    },
    {
        "@type": "simple",
        "tags": ["latitude"],
        "fieldDesc": "TransactionLatitude"
    },
    {
        "@type": "composite",
        "fields": ["TransactionLatitude","TransactionLongitude"],
        "tags": ["coordinate"],
        "fieldDesc": "TransactionCoordinates"
    }
]
```



To create the semantic file for your data source, you can download a template from Pulse. To do so, start creating a new AutoML run, select the data source, and use the **Download template file** option in the Semantic File field. Then simply add the tags to the fields.

The following are the **rules of tagging**:

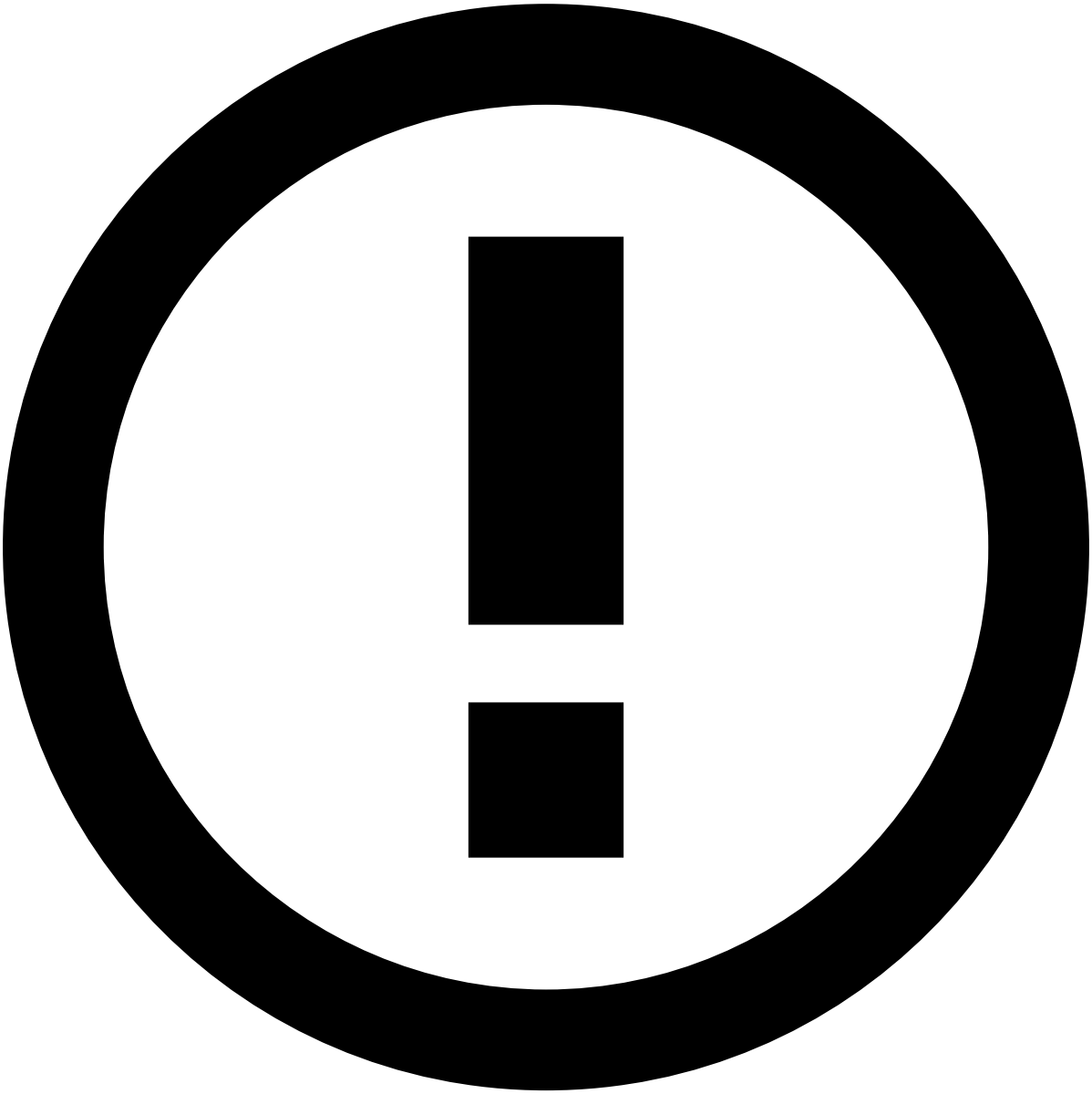
- At least one field must be tagged in the semantic file.
- Each field can appear only once.
- Each field must exist in the schema of the data source. However, complex data fields shouldn't be tagged, as they won't be used by AutoML.
- A composite tag must refer to fields that are tagged in the same file. For example, if you add the *coordinate* tag, in the same file there must be a field tagged with *longitude*, and one tagged with *latitude*, and the name of these fields must be listed in the *coordinate* tag, under *fields*.
- A contextual tag should be accompanied by the corresponding *grouping-entity* or *amount* tags.

List of input tagsâ□□

The following table lists the built-in tags of Pulse that can be used by Data Scientists in the semantic file when creating a new AutoML run, to tag the fields of the input data source.

Note that Pulse doesn't set the tags on its own. If the semantic file doesn't provide a tag for a field, Pulse cannot create features based on it.

In addition, the below file can be extended by developers with custom tags used by custom transformations.



Contextual tags

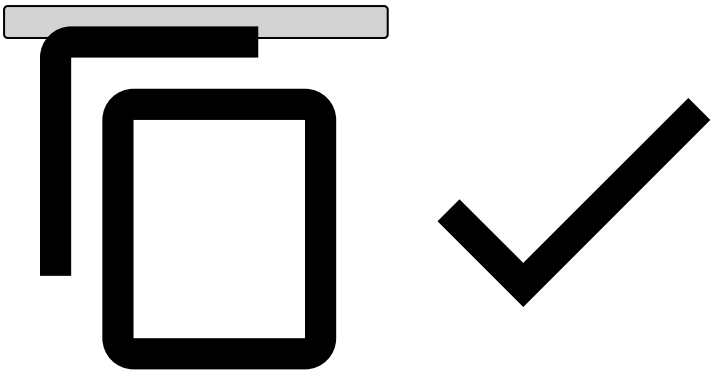
Contextual tags are tags that don't trigger any transformation *per se*. Theyare intended to be used by their companion tag.

For example, the Data Scientist might want to tag the field "card_id" as a *grouping-entity*. Additionally adding the contextual tag *card* will make AutoML only trigger transformations that make sense for cards.

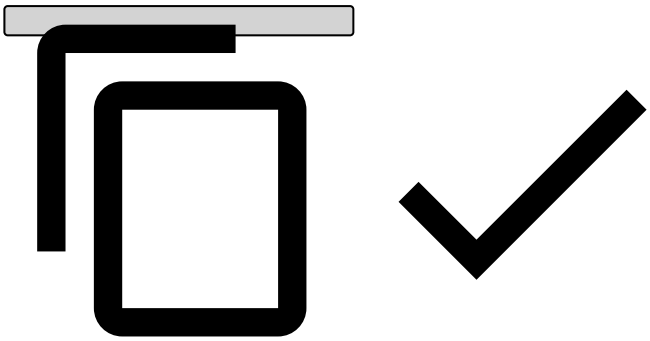
You can find existing contextual tags in the table below.

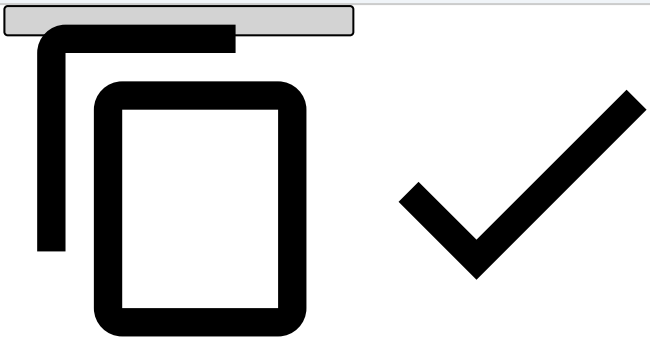
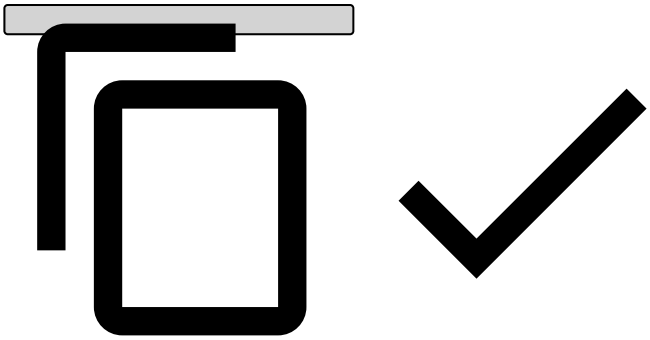
Usage example:

```
 {
   "@type": "simple",
   "tags": [
     "grouping-entity"
     "card"
   ],
   "fieldDesc": "card_id"
},
```

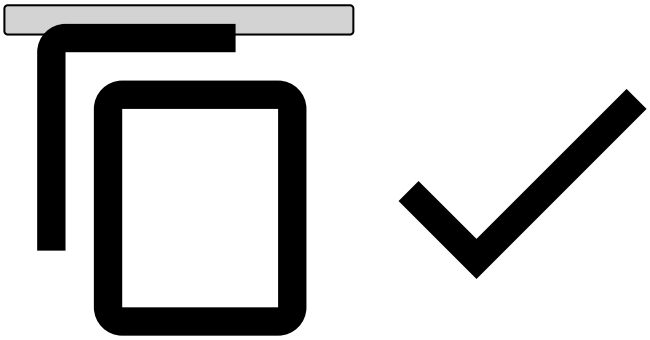


Type	Observation

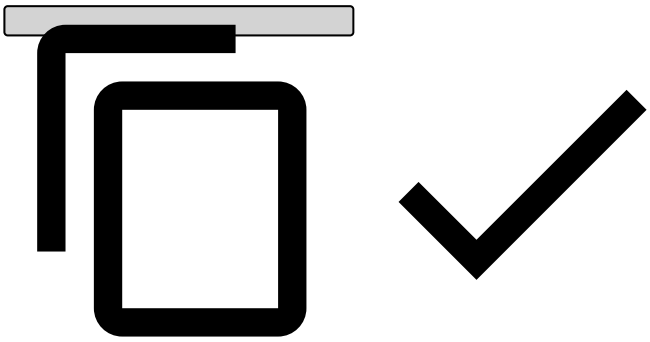
address	A composite tag that combines fields with line1, line2, zip, city, region and country tags. Example for tagging:
	
	`{ "@type": "composite", "fields": [ "CustomerAddressLine1", "CustomerAddressLine2", "CustomerAddressZipCode", "CustomerAddressCity", "CustomerAddressRegion", "CustomerAddressCountry" ], "tags": ["address"], "fieldDesc": "CustomerAddress" }`
	
	 
	
	To tag an address at least two components of the address must be present, so a minimal example would be:
	
	`{ "@type": "composite", "fields": [ "CustomerAddressLine1", "CustomerAddressZipCode", ], "tags": ["address"], "fieldDesc": "CustomerAddress" }`

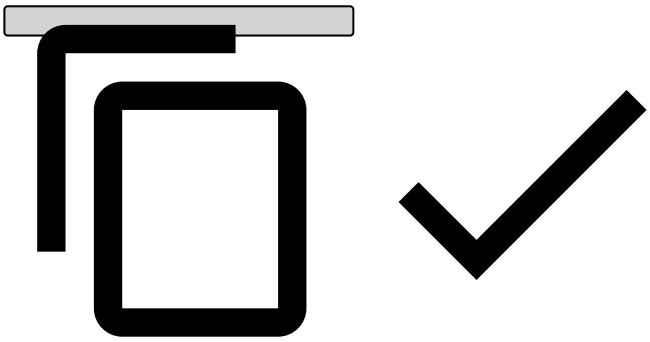
Type	Observation
	
line1	The first line of an address
line2	The second line of an address
zip	
city	
region	
country	Note: This tag can also be used as a contextual tag for the grouping-entity tag.
coordinate	A composite tag that combines fields with longitude and latitude tags. Example for tagging: `{ "@type": "composite", "fields": [ "TransactionLongitude", "TransactionLatitude" ], "tags": [ "coordinate" ], "fieldDesc": "TransactionCoordinates" }` 
longitude	A double, representing the longitude of a location
latitude	A double, representing the latitude of a location
location	This tag can be attributed to either countries or cities. Fields that have this tag will be compared. For example, if a data source has two fields that have the city and location tag, Pulse will generate a feature that checks if they match.
datetime	Tag for a field containing date and time in milliseconds, in Unix timestamp format
expiration-datetime	Tag for expiration dates, e. g. card expiration date.
yymm-datetime	Tag for fields of datetime with format yyMM.
name	Name of people, like the name on a card. It will be compared with the name in the email, for instance.

Type	Observation
email	
amount	

available-amount	Contextual tag for an amount field. Usage example:
	
	{
	  "@type": "simple",
	  "tags": [
	  "amount"
	  "available-amount"
	  ],
	  "fieldDesc": "available_credit"
	},
	
	 

entity	Tag for a field by which you want to order the data for aggregations to count distinct values.

grouping-entity	Tag for a field by which aggregations should be performed (i.e. we want to build profiles). Typical examples are cards, client ids or accounts. Typically a subset of the fields tagged with the entity tag. For example, imagine that we set the grouping-entity tag for card_id and client_id, and set the entity tag for card_id, client_id, user_phone and user_email. All the aggregations (averages, counts, sums, etc) will be performed by card, and also by client. Additionally, AutoML has features that count the number of distinct clients, emails and phones by card and distinct cards, emails and phones by client. Example for a composite field as grouping entity:
	
	{
	  "@type": "composite",
	  "fields": [
	  "customer_id",
	  "mcc"
	  ],
	  "tags": ["grouping-entity"],
	  "fieldDesc": "customer_id_mcc"
	},
	
	 

Type	Observation
	A composite grouping entity can work independently. If we want a composite grouping-entity with A, B, no need for A and B need to be tagged independently, which would create profiles for A, B and A+B.
acquirer, card, country, customer, ip, merchant, account	Contextual tags that make grouping-entities only trigger transformations that make sense according to that semantic. Usage example: `{ "@type": "simple", "tags": [ "grouping-entity" "card" ], "fieldDesc": "card_id" },` 
timestamp	A field of date and time value based on which the data can be aggregated for the metrics. Expected format: Unix timestamp.

Learn moreâ□□

- AutoML
- Creating features and models automatically